Petrocchi Gianmarco 599116

Progetto Farm

Progetto corso Sistemi Operativi e laboratorio; 09/01/2023

Premessa

Il progetto ***farm*** consiste nella realizzazione di un programma composto da due processi, il primo denominato *MasterWorker* e il secondo *Collector*. *MasterWorker* è un processo multi-threaded composto da un thread Master e da 'n' thread Worker, questo numero viene passato da riga di comando con l'argomento opzionale –n. Il programma prende come argomenti una lista di nomi di file binari contenenti numeri lunghi ed un certo numero di argomenti, che serviranno per poter configurare il *MasterWorker* e l'ambiente su cui andranno a lavorare i thread worker. Il processo *Collector* viene generato dal processo *MasterWorker.* I due processi comunicano attraverso una connessione socket AF_UNIX.
Il nome del generico file, che viene letto dal *MasterWorker*, viene inviato ad uno dei thread worker tramite una coda concorrente condivisa. Il thread worker si occupa di leggere dal disco il contenuto dell'intero file il cui nome ha ricevuto in input, e di effettuare un calcolo sugli elementi letti e quindi di inviare il risultato ottenuto, unitamente al nome del file, al processo *Collector* tramite la connessione socket precedentemente stabilita. Il processo *Collector* attende di ricevere tutti i risultati dai worker ed al termine stampa i valori ottenuti sullo standard output, ordinando la stampa in ordine crescente di risultato.
Gli argomenti che opzionalmente possono essere passati al processo MasterWorker sono i seguenti:

- -n <nthread> specifica il numero di thread worker del processo *MasterWorker (*valore di default 4*)*
- -q <qlen> specifica la lunghezza della coda concorrente (valore di default 8)
- -d <directory-name> specifica una directory in cui sono contenuti file binari ed eventualmente altre directory contenente file binari; i file binari dovranno essere utilizzati come file di input per il calcolo
- -t <delay> specifica un tempo in millisecondi che intercorre tra l'invio di due richieste successive ai thread worker da parte del thread Master (valore di default 0)

Il processo *MasterWorker* deve gestire i segnali SIGHUP, SIGINT, SIGQUIT, SIGTERM, SIGUSR1. Alla ricezione del segnale SIGUSR1 il processo *MasterWorker* notifica il processo *Collector* di stampare i risultati ricevuti sino a quel momento (sempre in modo ordinato), mentre alla ricezione degli altri segnali, il processo deve completare i task eventualmente presenti nella coda dei task da elaborare, non leggendo più eventuali altri file in input, e quindi terminare dopo aver atteso la terminazione del processo *Collector* ed effettuato la cancellazione del socket file. Il processo *Collector* maschera tutti i segnali gestiti dal processo *MasterWorker*. Il segnale SIGPIPE deve essere gestito opportunamente dai due processi.

Struttura del progetto

Il progetto è suddiviso in vari file, ognuno dei quali ha un proprio file header (.h) con i metodi e le variabili che necessita per il funzionamento:

- *Main.c:* file la cui unica funzionalità è quella di generare due processi tramite il metodo fork(). Il processo padre è il *MasterWorker*, il processo figlio è il *Collector.* Riceve in input i vari parametri da riga di comando, che verranno passati al processo padre per poter procedere con la configurazione.
- *Master_Thread.c:* processo padre che riceve in input i parametri passati da riga di comando. Nel caso in cui l'utente non passa nessun argomento tra quelli citati sopra, verrano utilizzati i valori di default che sono dichiarati all'interno del file *parameters.h.*Il *Master_Thread s*i occupa di: assegnare ai segnali che devono essere gestiti un handler, questo viene fatto subito in maniera tale da non incorrere in syscall che vengono interrotte; creare il socket per la comunicazione con il processo figlio (*Collector*); creare il pool di thread worker e la coda concorrente attraverso i parametri ricevuti; mandare a intervalli regolari di tempo, specificato dal parametro opzionale –t, i nomi dei file binari ai thread worker usando la coda creata.
- *Collector.c:* processo figlio che comunica con il *MasterWorker* attraverso il socket AF_UNIX. Riceve i nomi dei file binari e i risultati calcolati dai thread worker, questi vengono stampati in ordine crescente di risultato. Per poter garantire questo tipo di stampa, il *Collector* utilizza una linked list che inserisce gli

elementi in ordine crescente di risultato. L'implementazione di questo tipo di struttura dati è presente nel file *ascending_queue.c*

- *Thread_Worker.c:* thread che vengono creati dal *MasterWorker,* il quale passa a loro i parametri necessari: socket per la comunicazione con il *Collector;* puntatore alla coda concorrente per poter estrarre i nomi dei file; due puntatori a flag per la gestione dei segnali che vengono catturati dal *MasterWorker,* e a seconda del flag settato il thread worker comunicherà al *Collector* quello che dovrà fare. I thread worker si occupano di accedere concorrentemente alla coda per prelevare i nomi dei file e calcolare il risultato sulla base dei numeri lunghi letti. Il risultato e il nome del file vengono comunicati al *Collector* attraverso il socket. Per una corretta comunicazione l'accesso al socket (la scrittura) è concorrente, in quanto viene passato a tutti i thread worker lo stesso socket.
- *queue.c:* file contentente l'implementazione della coda concorrente che conterrà i nomi dei file. Per realizzare la coda è stata implementata una linked list con accesso concorrente di tipo FIFO (First In First Out). Per rendere le operazioni di inserimento ed estrazioni costanti (O(1)), la coda è stata realizzata attraverso due puntatori: uno per la testa, che viene manipolato solo se la lista è vuota (in caso di operazione di inserimento) e per l'estrazione; uno per la coda che viene manipolato in caso di operazioni di inserimento. Gli elementi di questa coda sono la coppia (filename, lunghezza del filename). A differenze delle classiche linked list questa ha una dimensione, la quale viene settata attraverso il parametro opzionale –q <qlen> passato attraverso CLI.
- *ascending_queue.*c: file contenente l'implementazione della coda che viene utilizzata dal processo *Collector* per stampare i risultati in ordine crescente. Gli elementi di questa coda sono la coppia (filename, result) .Questa lista non è concorrente, in quanto viene usata da un solo processo.
- *mystring.*c: file contenente alcuni metodi per la creazione e concatenazione di stringhe, utili per la creazione dei path assoluti dei file
- *utilities.*c: file contenente metodi per la lettura e scrittura su socket; metodi per il controllo dei tipi di parametri passati da CLI; metodi per il controllo di file e directory regolari.

Scelte implementative

- La terminazione del *MasterWorker* può avvenire prematuramente per diverse cause: è stato intercettato uno tra i segnali SIGINT, SIGQUIT, SIGHUP, SIGTERM; è avvenuto un errore in fase di configurazione dei parametri passati tramite CLI; è avvenuto un errore durante l'esecuzione di una syscall. Per quest'ultime due cause è stato implementato un metodo apposito, denominato "shutdown_master_thread_prematurely", che permette di liberare tutta la memoria allocata dal *MasterWorker* e di far terminare prematuramente il *Collector* mandando il segnale SIGUSR2 (Non presente nella traccia del progetto). Il Collector deve gestire quindi il segnale SIGUSR2, a tal proposito viene associata una funzione come handler, la quale si limita a settare un flag che permetterà di fare lo shutdown del collector e liberare la memoria allocata. La terminazione regolare del *MasterWorker* la si ha quando esso ha finito di inserire tutti i nomi di file specificati e presenti all'interno dell'eventuale cartella specificata con il parametro –d, attendendo la terminazione dei thread worker e anche del processo *Collector*. Quando il *MasterWorker* ha inserito tutti i file nella coda inserisce la stringa "finish", quando questa viene estratta da uno dei thread worker viene settato il flag *flag_finish* = true, in maniera tale da far terminare poi tutti i thread worker. Questa stringa speciale viene mandata tramite il socket al processo *Collector,* e quest'ultimo termina l'esecuzione stampando la lista ordinata dei risultati e liberando memoria.
- La coda concorrente, usata per l'inserimento e l'estrazione dei nomi dei file, è sincronizzata grazie all'uso di un mutex (denominato mutex_socket) e di due condition variables per mettere in attesa i thread del verificarsi di determinati eventi. Queste due condition variables sono associate rispettivamente: all'evento che la coda è vuota; all'evento che la coda è piena, in quanto quest'ultima ha una dimensione prefissata. Queste due condition variables sono fondamentali, in quanto l'utilizzo della coda segue il

paradigma producer-consumer: il producer è il processo *MasterWorker* che inserisce i nomi dei file nella coda, non può inserirli se la coda è piena; il consumer sono i thread worker generati dal *MasterWorker* che estraggono i nomi dei file dalla coda, non possono estrarli se la coda è vuota. Per questa coda viene usata anche un flag boolean denominato *"queue_closed",* usato per controllare se la coda è stata chiusa/distrutta. Viene fatto ciò perché può capitare che, nel momento in cui il *MasterWorker* attende la terminazione dei thread worker, la coda venga distrutta e quindi uno dei thread potrebbe controllare se la coda è vuota, generando una invalid read.

- L'accesso concorrente per la scrittura sul socket è garantito grazie all'uso di un mutex, questo viene acquisito prima di usare il socket e viene rilasciato subito dopo. Viene fatto ciò perché tutti i thread worker usano lo stesso socket per comunicare con il *Collector,* quindi non si ha un socket diverso per ogni thread.

Gestione segnali

Il processo *MasterWorker* e *Collector* devono gestire i segnali SIGINT, SIGHUP, SIGTERM, SIGQUIT, SIGUSR1 e SIGPIPE.
Questi segnali vengono mascherati dai due processi in maniere differenti: il processo *Collector* ignora tutti i segnali, quindi associa l'handler SIG_IGN a tutti i segnali; il processo *MasterWorker* associa una funzione, chiamata *signals_handler,* la quale setta uno dei due flag a seconda del segnale catturato. I due flag sono *flag_print* e *flag_others_signals,* entrambi volatile sig_atomic_t. Il primo flag viene settato quando viene catturato il segnale SIGUSR1, mentre il secondo flag viene settato quando viene catturato uno tra i segnali SIGINT, SIGQUIT, SIGTERM, SIGHUP. Il segnale SIGPIPE viene ignorato da entrambi i processi, quindi verrà settato SIG_IGN come handler.
L'aggiornamento dei due flag citati poc'anzi, viene visto dai thread worker i quali comunicheranno al processo *Collector* ciò che dovrà fare attraverso la socket. Se viene intercettato il segnale SIGUSR1 (*flag_print* settato), il thread worker scrive sulla socket la stringa "print", la quale indica al *Collector* di stampare la lista ordinata dei risultati. La cattura di questo segnale non implica la terminazione del processo *MasterWorker* né tanto meno del processo *Collector.*
Se viene intercettato uno degli altri segnali, associati al flag *flag_others_signals,* il processo *MasterWorker* inserirà nella coda concorrente la stringa "finish", la quale indica di aver terminato l'inserimento di nomi dei file e che è prossimo alla terminazione. In questo modo i thread worker termineranno l'esecuzione una volta che uno di essi ha estratto dalla coda questa stringa speciale, e quest'ultima viene inviata tramite la socket al processo *Collector* che provvederà a stampare la lista ordinata finale e terminerà liberando memoria e chiudendo la socket.

Test di esecuzione

I test sono presenti nel Makefile, al suo interno ci sono 3 target: "test" che esegue lo script test.sh; "test1" e "test2" che eseguono rispettivamente lo script test1.sh e test2.sh presenti nella cartella script. Per eseguirli basta posizionarsi all'interno della cartella dove c'è Makefile e digitare da CLI "make <nome target>".
Di seguito saranno mostrate alti test di esecuzione, riguardo a casistiche non comprese negli script bash.

```
osboxes@osboxes:~/Desktop/ProgettoSOL$ ./farm file1.dat file2.dat file3.dat file4.dat file5.dat file6.dat file7.da
Processo padre 25323
Will use default value for option -q
Will use default value for option -n
Will use default value for option -t
No directory name set from command line
Processo figlio 25324
file6.dat isn't a regular file
file7.dat isn't a regular file
Final result
103453975 file2.dat
153259244 file1.dat
293718900 file3.dat
380867448 file5.dat
584164283 file4.dat

Processo figlio 25324 terminato con successo
Processo padre 25323 terminato con successo
```

1. Nessun parametro viene passato da riga di comando, vengono usati i valori di default

```
osboxes@osboxes:~/Desktop/ProgettoSOL$ ./farm -n pippo -q 2  file1.dat file2.dat file3.dat file4.dat file5.dat file
6.dat file7.dat
Processo padre 25361
Option -n require an integer parameter
Processo figlio 25362 terminato
Processo padre 25361 terminato con fallimento
```

2. Passaggio di parametro errato, -n necessita un numero intero. In questo caso viene passata una stringa. Questa tipologia di esecuzione avviene anche in altri casi: parametro –n numero negativo; parametro –q numero minore o uguale a 0 oppure non numero; parametro –t non un numero oppure numero negativo

```
osboxes@osboxes:~/Desktop/ProgettoSOL$ ./farm -n 1 -q 2 -d Esempi file1.dat file2.dat file3.dat file4.dat file5.dat
 file6.dat file7.dat
Processo padre 25450
Option -d require a legit directory name
Processo figlio 25451 terminato
Processo padre 25450 terminato con fallimento
```

3. Passaggio di un nome di directory non esistente/valido

```
osboxes@osboxes:~/Desktop/ProgettoSOL$ ./farm -n 2 -q 3 file* -d testdir -t 1000 &
[1] 25696
osboxes@osboxes:~/Desktop/ProgettoSOL$ Processo padre 25696
Value for option -q setted
Value for option -n setted
Value for option -t setted
Value for option -d setted
Processo figlio 25697
kill -10 25696
osboxes@osboxes:~/Desktop/ProgettoSOL$ SIGUSR1 communication:
64834211 file100.dat
258119464 file116.dat
532900090 file117.dat
1146505381 file10.dat
1485251680 file12.dat
1674267050 file13.dat

kill -2 25696
osboxes@osboxes:~/Desktop/ProgettoSOL$ Final result
64834211 file100.dat
258119464 file116.dat
518290132 file17.dat
532900090 file117.dat
748176663 file16.dat
1146505381 file10.dat
1485251680 file12.dat
1674267050 file13.dat
1878386755 file14.dat
2086317503 file15.dat
2322416554 file18.dat

Processo figlio 25697 terminato con successo
Processo padre 25696 terminato con successo
```

4. Viene mandato il segnale SIGUSR1 (kill -10 25696) e poi il segnale SIGINT (kill -2 25696)

```
osboxes@osboxes:~/Desktop/ProgettoSOL$ ./farm -n 2 -q 3 file* -d testdir -t 1000 &
[1] 25708
osboxes@osboxes:~/Desktop/ProgettoSOL$ Processo padre 25708
Value for option -q setted
Value for option -n setted
Value for option -t setted
Value for option -d setted
Processo figlio 25709
kill -10 25708
osboxes@osboxes:~/Desktop/ProgettoSOL$ SIGUSR1 communication:
64834211 file100.dat
258119464 file116.dat
532900090 file117.dat
1146505381 file10.dat
1485251680 file12.dat
1674267050 file13.dat
1878386755 file14.dat
2086317503 file15.dat

kill -10 25708
osboxes@osboxes:~/Desktop/ProgettoSOL$ SIGUSR1 communication:
64834211 file100.dat
153259244 file1.dat
258119464 file116.dat
518290132 file17.dat
532900090 file117.dat
748176663 file16.dat
1146505381 file10.dat
1485251680 file12.dat
1674267050 file13.dat
1878386755 file14.dat
2086317503 file15.dat
2322416554 file18.dat

Final result
64834211 file100.dat
103453975 file2.dat
153259244 file1.dat
193031985 testdir/testdir2/file150.dat
258119464 file116.dat
293718900 file3.dat
380867448 file5.dat
518290132 file17.dat
532900090 file117.dat
584164283 file4.dat
748176663 file16.dat
```

```
890024017 testdir/file19.dat
985077644 testdir/file8.dat
1146505381 file10.dat
1485251680 file12.dat
1674267050 file13.dat
1878386755 file14.dat
1884778221 testdir/testdir2/file111.dat
2086317503 file15.dat
2322416554 file18.dat
2560452408 file20.dat

Processo figlio 25709 terminato con successo
Processo padre 25708 terminato con successo
```

5. Viene mandato più volte il segnale SIGUSR1 e si attende la terminazione senza mandare altri segnali